MagicQ miniQ Encoders (dual module)

A custom hardware controller for the virtual encoders of **ChamSys MagicQ**, built from **two Raspberry Pi Pico (RP2040 Zero)** – a **left** and a **right** module, each carrying **4 rotary encoders with push button**.

	Left module	**Right module**
On-screen encoders it controls	1 – 4	5 – 8
Encoders	4	4
Extra keys	none	none
Pinout	identical to the right module	identical to the left module

Both modules are separate USB-HID devices (each emulates an **absolute mouse + relative mouse** composite), so no drivers are needed. Every physical knob first moves the mouse cursor onto its matching **on-screen encoder** and then scrolls or clicks exactly there — no key codes, no manual cursor positioning. The **graphical attribute keys** (INT/POS/COL/BEAM/...) are not part of this device anymore; they live on the **Wing** (row 3).

> **Note:** Works in **any** MagicQ keyboard mode, including **Playback shortcuts** (which the wing needs). No CAPS LOCK / mode toggling is used.

Table of contents

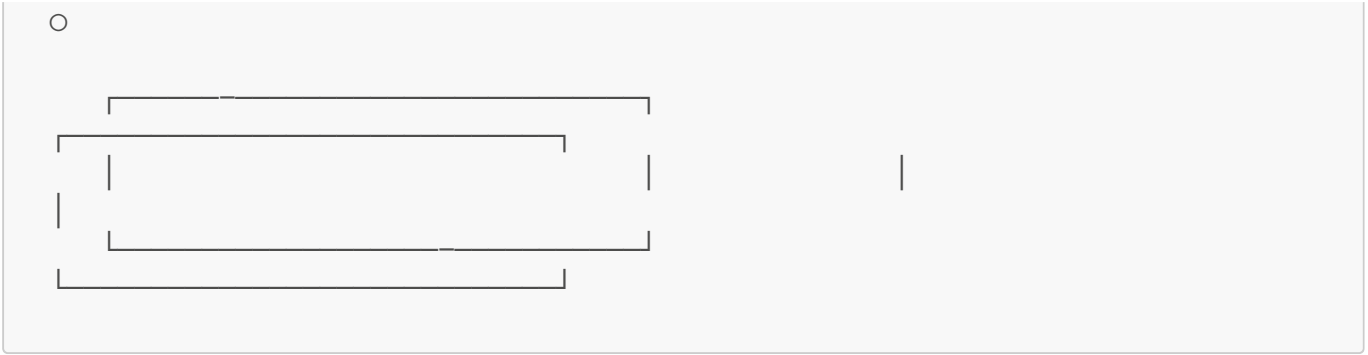
- 1. The layout (physical arrangement)
  - 1.1 Control mapping (verified)
  - 1.2 Per-module cursor calibration (no re-flashing needed)
- 2. Required hardware & shopping list
- 3. Hardware wiring
  - 3.1 Encoder direct connection (Pico GPIO)
  - 3.2 Connection diagram (ASCII)
- 4. Build & upload (PlatformIO)
- 5. Setup in MagicQ
- Notes & limitations
- Project structure

1. The layout (physical arrangement)

Each module (left and right) carries **4 rotary encoders with click**:

```
   [ LEFT MODULE ]  = On-screen encoders 1–4    [ RIGHT MODULE ]  = On-
screen encoders 5–8

   (Enc 1)  (Enc 2)  (Enc 3)  (Enc 4)          (Enc 5)  (Enc 6)  (Enc 7)
(Enc 8)
     ○        ○        ○        ○               ○        ○        ○
```



- Each module uses the **same firmware** and the **identical pinout** – only the compiled-in *default* targets differ (`ENC_SIDE=0` left = encoders 1–4, `ENC_SIDE=1` right = 5–8).
- There are **no other buttons** on the modules (no attribute keys, no expander).
- The 4 encoder **switches** sit directly on the Pico GPIOs (against GND, pull-ups enabled).

1.1 Control mapping (verified)

Control	Action	HID output
Encoder **right**	Cursor to on-screen encoder + wheel up (value +)	abs. mouse + wheel +1*
Encoder **left**	Cursor to on-screen encoder + wheel down (value –)	abs. mouse + wheel –1*
Encoder **click** (SW)	Cursor to on-screen encoder + click	abs. mouse + left click

** Wheel direction is the verified behaviour of the previous wing. If your wiring counts the other way, set `#define ENC_INVERT_DIR 1` in `src/main.cpp` (no pin swap needed).*

> **How the encoders work:** MagicQ on PC/Mac shows its **on-screen encoders** in the console layout (full screen / maximized). The physical knobs first move the **mouse cursor** to the matching on-screen encoder (absolute positioning, see `encCal.x/y`) and then send a **mouse wheel** scroll or left click exactly there. The attribute windows (INT/POS/COL/BEAM), FX and GROUP shortcuts are handled by the **Wing** (row 3), not by this device.

1.2 Per-module cursor calibration (no re-flashing needed)

Each of the 4 on-screen encoders of a module has its **own** target position (no interpolation). The values live in `encCal.x[4]` / `encCal.y[4]` (`src/main.cpp`) and are stored in the module's own LittleFS flash as **`/ecal.bin`**. The calibration is therefore **per RP2040** – both modules are calibrated independently, each against its own half of the MagicQ console layout.

Restart the module while holding down encoder 1 (its switch) → the calibration UI starts automatically. The mouse cursor follows the selected encoder, so you can see what you are moving.

Action	Function
Click encoder *i*	select encoder *i* (cursor jumps to its stored position)
Turn encoder 1 / 2	selected encoder: **X** coarse / fine (±500 / ±50)

Action	Function
Turn encoder 3 / 4	selected encoder: **Y** coarse / fine (±500 / ±50)
Click encoder *i* again	**save** that encoder's position — the LED blinks the number of encoders saved so far
all 4 saved	`/ecal.bin` is written and the module **restarts automatically** — this is the only way out of calibration

Turn clockwise = plus (right / down). Encoders 1–4 act purely as adjustment knobs while calibrating; the target positions always come from the encoder SW clicks. `[CAL] …` on the serial console reports every selection, adjustment and save.

> **Note:** The calibration values are stored in LittleFS, which requires the **64 KB filesystem partition** (`board_build.filesystem_size = 64k`, already set in `platformio.ini`). Values are read back on every boot (`[CAL] /ecal.bin geladen: …`, otherwise the compiled-in defaults for the selected side).
>
> **The unit suffix matters:** the Raspberry Pi platform parses a **bare number as bytes**, so `64` means 64 *bytes*, not 64 KB. A 64-byte partition makes the LittleFS format loop forever with interrupts disabled, so the module hangs at boot and never enumerates. Always write `64k`.

2. Required hardware & shopping list

For the **complete console** (left + right module):

Qty	Component	Note
2	Raspberry Pi Pico (RP2040 Zero)	1× left, 1× right module; onboard WS2812 on GP16 (e.g. RP2040-Zero "WS")
8	Mech. rotary encoders with push button (e.g. EC11)	4 per module, 20 detents, 3-pin variant
–	hook-up wires / stranded wire	for wiring
–	optional: pin headers / breadboards	

> **Pico pin usage per module (12 of 30 GPIOs):**
>
> - **GP29/28/27/26** = encoder A (CLK) 1–4
> - **GP3/4/5/6** = encoder B (DT) 1–4
> - **GP15/7/14/8** = encoder switches 1–4 (against GND, pull-up)
> - **GP16** = onboard WS2812 (NeoPixel) LED — flashes green on each action
> - All remaining GPIOs are free.

3. Hardware wiring

3.1 Encoder direct connection (Pico GPIO)

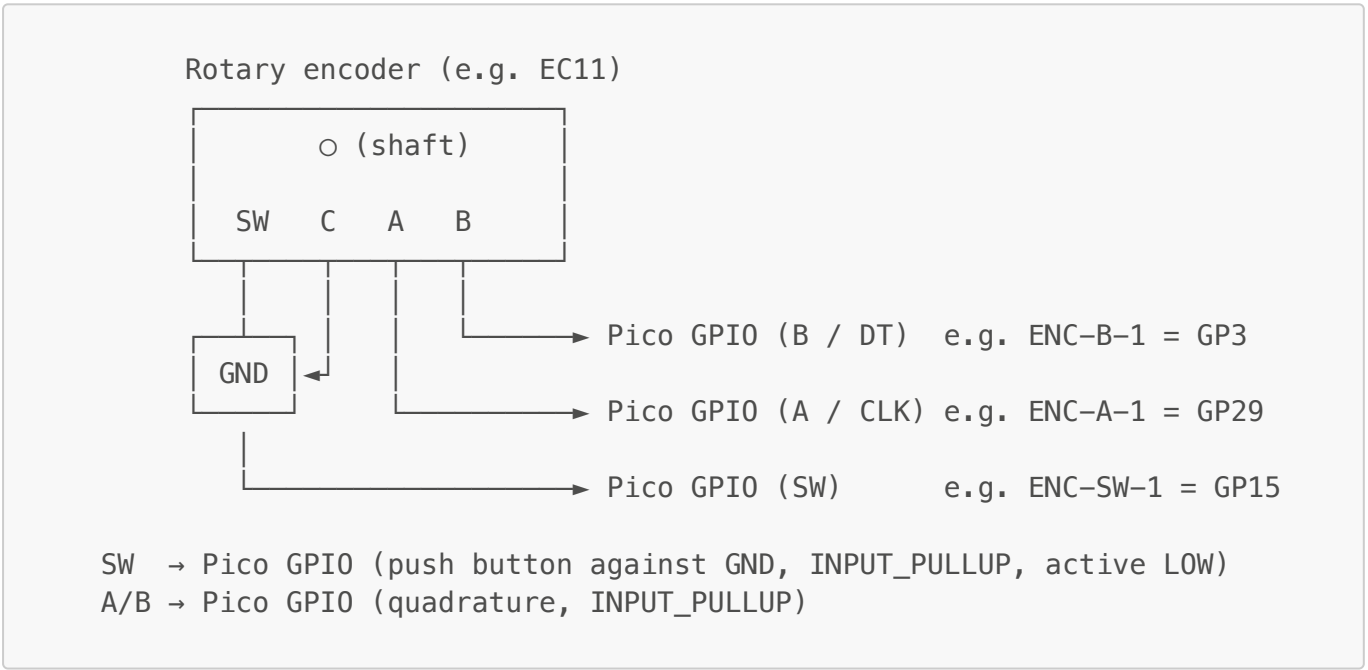
Both modules use the **identical** wiring:

Component	A (CLK)	B (DT)	SW (button)
Encoder 1	GP29	GP3	GP15
Encoder 2	GP28	GP4	GP7
Encoder 3	GP27	GP5	GP14
Encoder 4	GP26	GP6	GP8

Each switch goes **against GND** (the firmware enables internal pull-ups; SW = LOW means pressed).

3.2 Connection diagram (ASCII)

Rotary encoder (e.g. EC11) with integrated push button (SW); **C to GND**:



If a module's encoders scroll in the wrong direction, either swap A/B on the board or flip `ENC_INVERT_DIR` in the firmware (recommended).

4. Build & upload (PlatformIO)

Prerequisite: PlatformIO Core installed.

```
# Switch to the project directory
cd Enncoder

# Build the left module (default)  /  the right module
pio run -e pico-left                # -DENC_SIDE=0  (on-screen encoders 1-4)
pio run -e pico-right               # -DENC_SIDE=1  (on-screen encoders 5-8)

# Compile + upload (both modules)
```

```
pio run -e pico-left  -t upload     # left module
pio run -e pico-right -t upload     # right module

# Serial console (debug output)
pio device monitor
```

Important: Uploads use the RP2040's native **1200-bps CDC touch** (core `SerialUSB` + `use_1200bps_touch` in the `rpipico` board): `pio run -t upload` reboots the running device into BOOTSEL by itself, so **no BOOTSEL button** is needed after the first flash. Only the **first flash** of each Pico requires bootloader mode via **BOOTSEL** (hold, plug in USB, release).

`-DENABLE_PICOTOOL_USB` must **not** be set: it makes the running firmware advertise the Raspberry Pi USB reset interface, but on this composite HID stack it broke USB enumeration on macOS (no `/dev/cu.usb*` port, no HID mouse).

`ENC_SIDE` only sets the compiled-in **default** target positions. After the per-module calibration (section 1.2) both modules behave correctly regardless of the side you flashed — each stores its own `/ecal.bin`.

> **Note on libraries:** The `Rotary` library comes from `lostcaggy/CaggyRotary` (declared in `platformio.ini`), and `adafruit/Adafruit NeoPixel` drives the onboard WS2812 on GP16. The `lib_ldf_mode = chain+` option builds the required core library `tusb-hid` automatically. **No** external TinyUSB package and **no** MCP23017/I²C library are needed anymore.
>
> `board_build.filesystem_size = 64k` reserves the 64 KB **LittleFS** partition that the calibration needs (a bare `64` would mean 64 *bytes* and hang the boot — always use the `k`).

5. Setup in MagicQ

The absolute-mouse encoder control works in **any** MagicQ keyboard mode, no mode switch is required.

1. Start MagicQ **full screen / maximized** so the on-screen encoders of the console layout appear at fixed screen positions.
2. Connect **both** modules (left + right) via USB — each is a separate HID device.
3. Run the position calibration once per module (section 1.2): the **left** module targets on-screen encoders **1–4**, the **right** module encoders **5–8**.
4. If MagicQ runs maximized on another resolution or window size, re-run the calibration of the affected module.

Then every turn/click of a knob moves the cursor to the matching on-screen encoder and changes that attribute. This works alongside the **Wing** (including its **Playback shortcuts** mode).

Notes & limitations

- **Only encoders:** Each module has just the 4 rotary encoders with push button — no attribute keys, no modifier keys, no I/O expander. All 4 switches sit directly on the Pico GPIOs.
- **Identical firmware:** Both modules use one firmware + one pinout. `ENC_SIDE` selects only the default targets; the **per-module calibration** (`/ecal.bin`) fully determines the mapping.
- **Absolute-mouse encoders:** The 4 knobs work as an **absolute mouse** (positioning + wheel) plus a **relative mouse** as the first HID report (macOS-compatible composite). Turning/clicking a knob

first positions the cursor on the matching on-screen encoder (`encCal.x/y`) and then scrolls/clicks there. Works in every keyboard mode and needs no CAPS LOCK / mode toggling. Positions are calibrated at runtime and stored in `/ecal.bin`.

- **USB-HID:** Each module appears to the PC as an **absolute mouse + relative mouse** (composite HID, **no keyboard**). Mouse reports are registered before any keyboard reports on a composite — macOS otherwise ignores pointing devices.
- **Wing note:** The graphical attribute windows (INT/POS/COL/BEAM), GROUP, FX and SHIFT now sit on the **Wing** (row 3 of the 5×13 matrix).
- The **serial output (115200)** is only for debugging and is not required for operation.

Project structure

```
Enncoder/
├── README.md              # This handbook
├── README.pdf             # PDF export of this handbook
├── platformio.ini         # PlatformIO configuration (pico-left / pico-
right)
└── src/
    └── main.cpp           # Firmware (left + right module share this file)
```

> The encoder PCB design sources are kept in the shared `../easyEDA/` project.